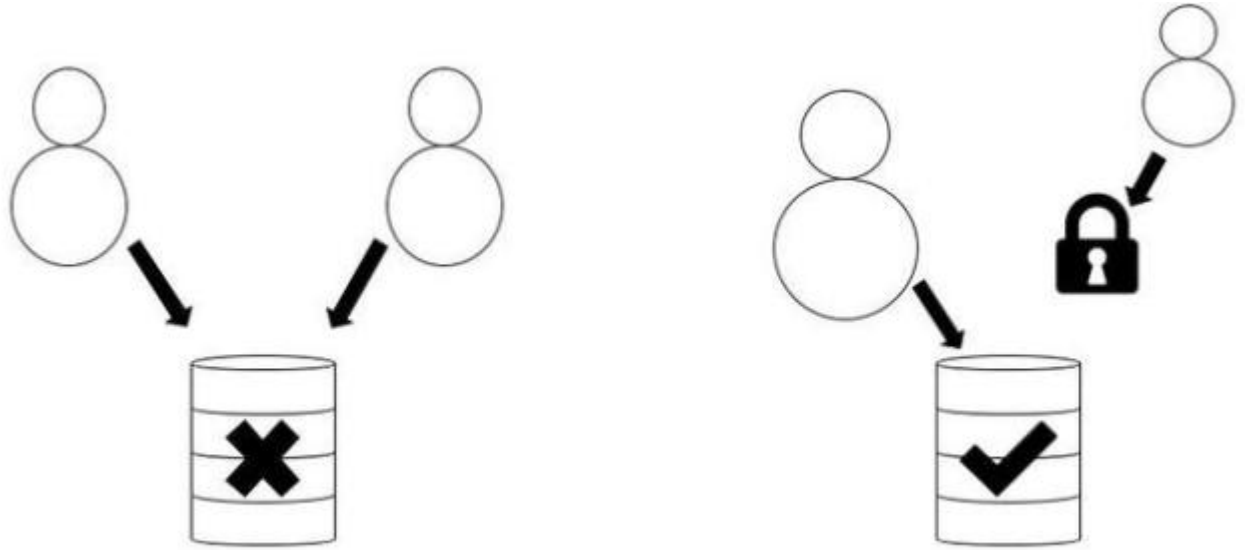
UNIT 3: Part II
Concurrency Control

Concurrency Control

- Concurrent Execution of transactions is unavoidable as it gives better throughput.
- Control of concurrent transactions must be taken care by the system through concurrency control scheme.
- There are set of protocols that will ensure serializability of schedules.
- The most popular and simple one is locking technique.

Lock

- *A Lock is a variable assigned to any data item in order to keep track of the status of that data item so that isolation and non-interference is ensured during concurrent transactions.*
- At its basic, a database lock exists to prevent two or more database users from performing any change on the same data item at the very same time.
- In layman's terms, this may be further simplified to the metaphorical 'lock' that is put on a data item so that no other user may unlock the ability to perform any update query.



Case 1: Simultaneous access to transactions is not permitted. Case 2: putting a lock on other transactions is feasible

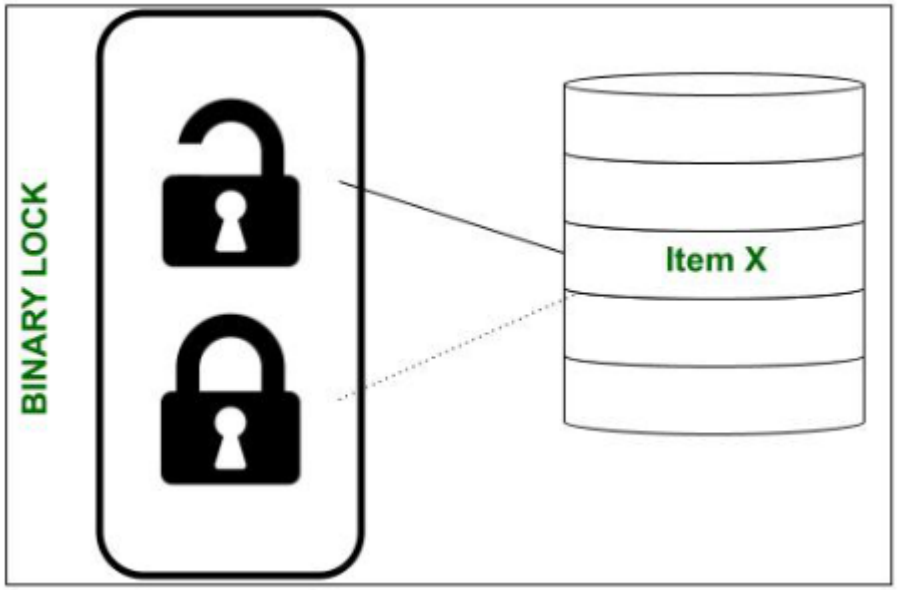
- In Case 2, which has been portrayed above, if the user/session on the right attempts an update, it will be met with a **LOCK WAIT** state or otherwise be **STALLED** until access to the data item is unlocked. In some situations – if the stall exceeds a time limit – the session is terminated and an error statement is returned.

Lock based Concurrency Control

- 2 types are there.
  - Binary
  - Shared & Exclusive
- **Binary Locks:** A Binary lock on a data item can either locked or unlocked states.
- **Shared/exclusive:** This type of locking mechanism separates the locks in DBMS based on their uses. If a lock is acquired on a data item to perform a write operation, it is called an exclusive lock.
- **1. Shared Lock (S):**
- A shared lock is also called a Read-only lock. With the shared lock, the data item can be shared between transactions. This is because you will never have permission to update data on the data item.
- For example, consider a case where two transactions are reading the account balance of a person. The database will let them read by placing a shared lock. However, if another transaction wants to update that account's balance, shared lock prevent it until the reading process is over.
- **2. Exclusive Lock (X):**
- With the Exclusive Lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item. X-lock is requested using lock-x instruction. Transactions may unlock the data item after finishing the 'write' operation.
- For example, when a transaction needs to update the account balance of a person. You can allows this transaction by placing X lock on it. Therefore, when the second transaction wants to read or write, exclusive lock prevent this operation.

Binary Lock

- Remember that a lock is fundamentally a variable which holds a value. A binary lock is a variable capable of holding only 2 possible values, i.e., a **1 (depicting a locked state)** or a **0 (depicting an unlocked state)**. This lock is usually associated with every data item in the database (maybe at table level, row level or even the entire database level).



- Should item X be unlocked, then a corresponding object lock(X) would return the value 0. So, the instant a user/session begins updating the contents of item X, lock(X) is set to a value of 1. Due to this, for as long as the update query lasts, no other user may access the item X – even read or write to it!
- There are 2 operations used to implement binary locks. They are lock_data( ) and unlock_data( ).

Binary Lock

- LOCK(X) – lock on object X => L(X) = 1
- UNLOCK(X) – Unlock on object X => L(X) = 0

Algorithm:

```
LOCK(X)
{
        if L(X)=0
       then L(X)=1
    else                  //when lock(x)==1 or item(x)  is locked
       wait until L(x)=0      // wait for the user to finish the update query
       wake up T
}
```

Binary Lock

Algorithm:

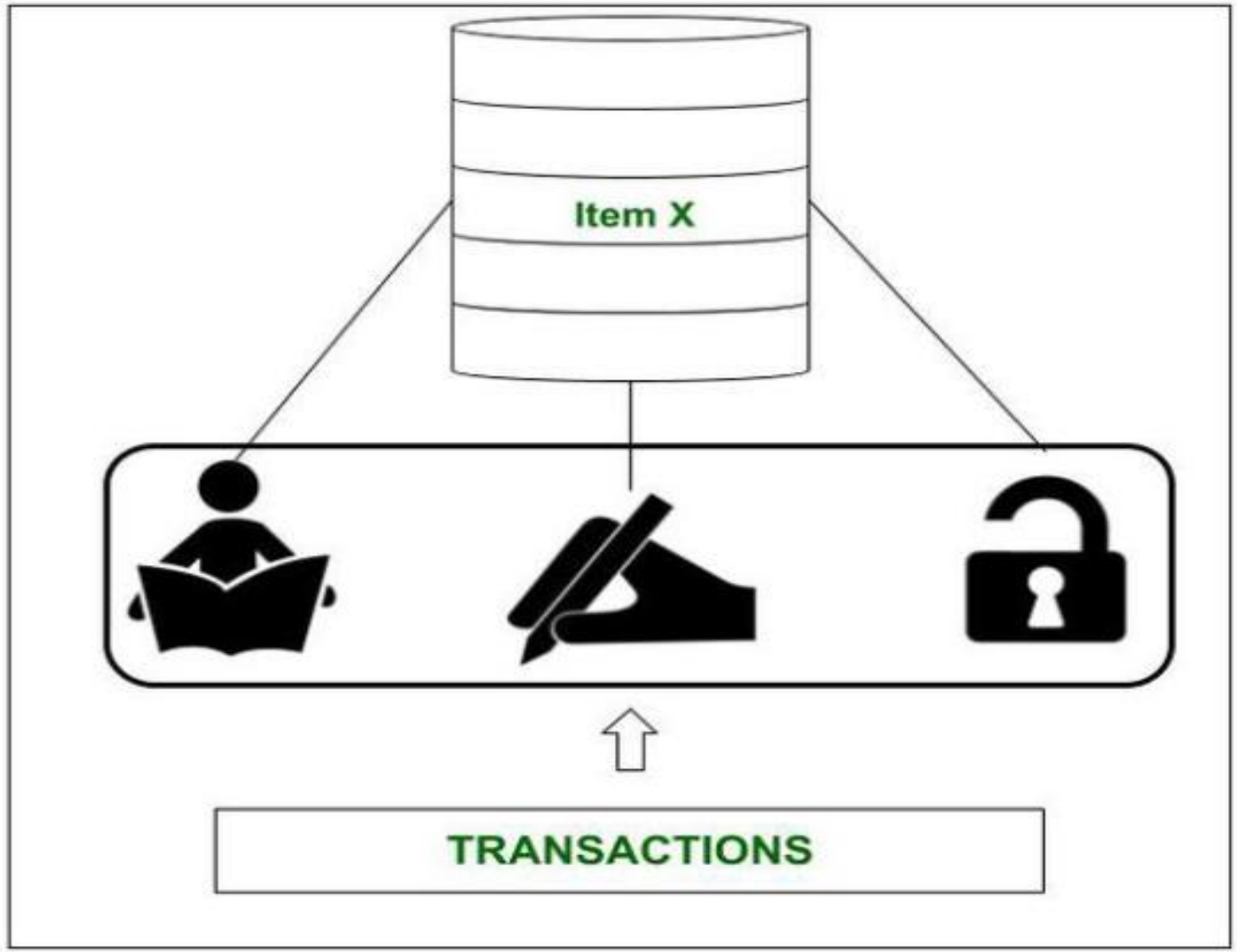
```
unlock_data(X):
    lock(X) = 0 //we unlock access to item X
    if (transactions are in queue)
    {
        then grant access or 'wake' the next transaction in line;
    }
```

Shared & Exclusive Lock

- In case of binary lock, even if the read-read operation occurs, there is a lock getting applied.
- But R-R is not a conflict.
- Hence we use the concept of shared lock & exclusive lock.

- **Shared or Exclusive Locks :**
  The incentive governing these types of locks is the restrictive nature of binary locks. Here we look at locks which permit other transactions to make read queries since **a READ query is non-conflicting**.
- However, if a transaction demands a write query on item X, then that transaction must be given exclusive access to item X. So, we require a kind of **multi-mode lock** which is what shared/exclusive locks are. They are also known as Read/Write locks.

- Unlike binary locks, Read/Write locks may be set to 3 values, i.e., **SHARED**, **EXCLUSIVE** or **UNLOCKED**. Hence, our lock, i.e., lock(X), may reflect either of the following values:
- **READ-LOCKED –**
  If a transaction only requires to read the contents of item X and the lock only permits reading. This is also known as a *shared lock*.
- **WRITE-LOCKED –**
  If a transaction needs to update or write to item X, the lock must restrict all other transactions and provide exclusive access to the current transaction. Thus, these locks are also known as *exclusive locks*.
- **UNLOCKED –**
  Once a transaction has completed its read or update operations, no lock is held and the data item is unlocked. In this state, the item may be accessed by any queued transactions.



A Shared/Exclusive lock may hold any of the 3 states.

Shared Lock

- If a shared lock is requested by T, the lock manager grants it, provided there is no exclusive lock on the object A. ie L(A) = s
- Lock manager the increments the count of number of read locks on A by 1.

Exclusive Lock

- If an exclusive lock is requested by T, the lock manager grants it provided no other transactions hold the lock on A.
- If the request raised by T cannot be granted, then T is inserted into the waiting queue.

Shared Lock Algorithm

```
S(A)
{
        if L(A) =0      //no lock on A
      then {
        L(A) = S  // set initial shared lock
        N(A) = 1  // s lock counter
        update lock table  }
    else
      if L(A) = S  //Already shared
      then {
        N(A) = N(A) + 1
        update lock table  }
      else insert T into queue & suspend T //A has an exclusive lock
}
```

Shared Lock Algorithm unlocking

```
U(A)
{
        if L(A) = 'X'
      then {
        L(A) = 0  // release the lock
        Activate a T from queue }
    else if L(A) = 'S'
      then {
        N(A) = N(A) - 1
        if N(A) = 0
          then L(A) = 0
            activate a T from queue}
}
```

Exclusive Lock Algorithm

```
X(A)
{
        if L(A) =0
      then {
        L(A) = 'X'  //set exclusive lock
        update lock table  }
    else     //already locked
        Insert T into queue
}
```

- A transaction obtaining a shared mode lock on a data item can only read the data item. It cannot write or update.
- A transaction obtaining exclusive mode lock can both read and write the data item.
- Several shared mode locks can be held simultaneously on a data item by different transactions.

Lock compatibility matrix

	U	S	X
U	-	√	√
S	√	√	
X	√		

- Here are a few rules that Shared/Exclusive Locks must obey:
- A transaction **T** MUST issue the unlock(X) operation after all read and write operations have finished.
- A transaction **T** may NOT issue a read_lock(X) or write_lock(X) operation on an item which already has a read or write-lock issued to itself.
- A transaction **T** is NOT allowed to issue the unlock(X) operation unless it has been issued with a read_lock(X) or write_lock(X) operation.

- **Drawbacks of Shared/Exclusive Locks :**
- Do not guarantee serializability of schedules on their own. A separate protocol must be followed to ensure this.
- Commercially not optimized for speedy transactions; not the best solution due to lock contention issues.
- Performance overhead is not negligible.

Two- Phase Locking

Two Phase Locking Protocol also known as 2PL protocol is a method of concurrency control in DBMS that ensures serializability by applying a lock to the transaction data which blocks other transactions to access the same data simultaneously. Two Phase Locking protocol helps to eliminate the concurrency problem in DBMS.

This locking protocol divides the execution phase of a transaction into three different parts.

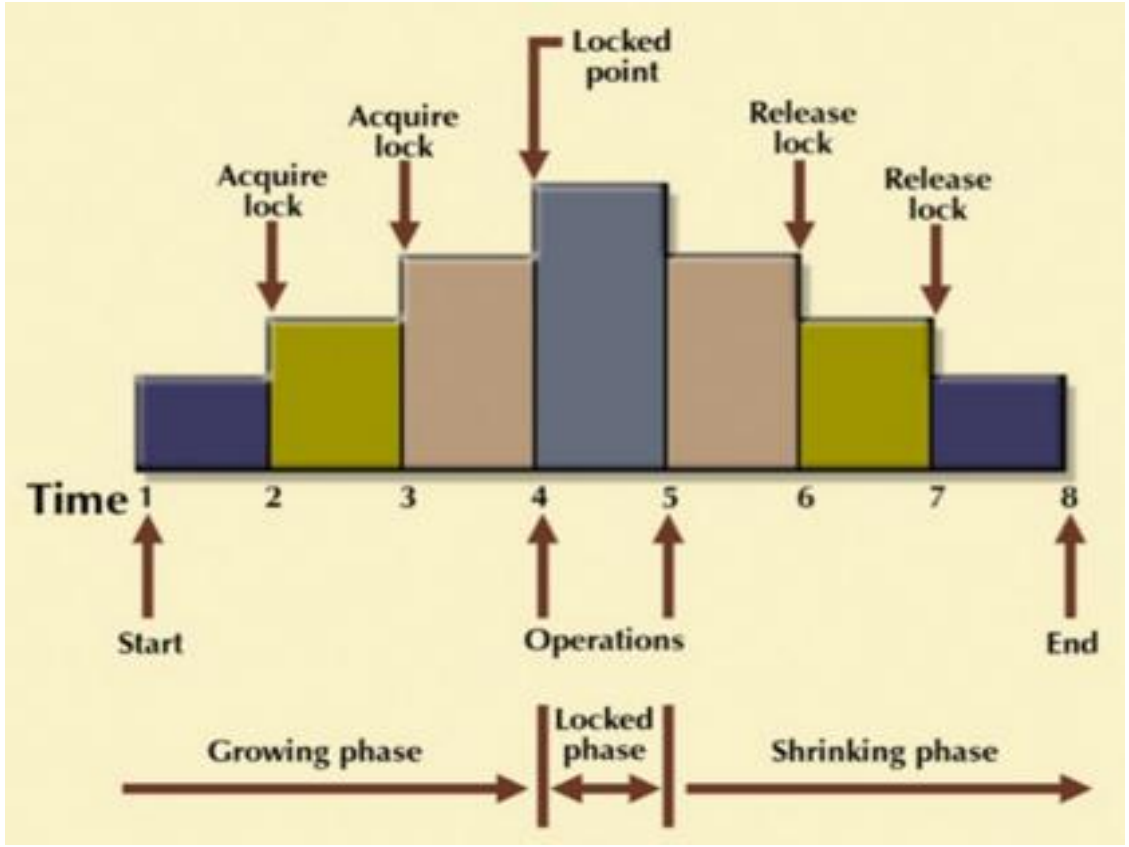
In the **first phase**, when the transaction begins to execute, it requires permission for the locks it needs.

The **second part** is where the transaction obtains all the locks. When a transaction releases its first lock, the third phase starts.

In this **third phase**, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.

Two- Phase Locking

- Two-phase locking protocol
  - All locking operations precede the first unlock operation in the transaction
  - Phases
    - Expanding (growing) phase
      - New locks can be acquired but none can be released
      - Lock conversion upgrades must be done during this phase
    - Shrinking phase
      - Existing locks can be released but none can be acquired
      - Downgrades must be done during this phase



It is true that the 2PL protocol offers serializability. However, it does not ensure that deadlocks do not happen.

Example of Two-Phase Locking

Consider two transactions, T1 and T2, both needing to access and update data items A and B.

- **Transaction T1 starts :**
- T1 requests a shared (read) lock on A.
- T1 requests a shared (read) lock on B.
- T1 enters its shrinking phase and releases the lock on A.
- T1 releases the lock on B.
-
- **Transaction T2 starts:**
- T2 requests a shared (read) lock on B.
- T2 requests a shared (read) lock on A.

- Conflict Scenario: If T1 requests to write to A, it would need an exclusive lock, and T2 would have to wait for the shared lock on A to be released by T1. T1 requests a write lock on B, and T2 waits for T1 to release the lock.
- Outcome: T1 would acquire a read lock on A, then a read lock on B, enter its lock point, then release the lock on A and B in its shrinking phase. T2 would then be able to acquire a read lock on A and a read lock on B

Potential Issues

- Deadlock: In some scenarios, like the example above, T1 and T2 could become deadlocked, where each transaction is waiting for a lock held by the other, preventing either from proceeding.
- Cascading Rollbacks: A rollback in one transaction can trigger rollbacks in other transactions that are dependent on the failed transaction, leading to data inconsistencies.
- Reduced Concurrency: 2PL can limit the number of concurrent transactions, potentially leading to decreased performance.

Variations of Two-Phase Locking

- Strict 2PL
  - Transaction does not release exclusive locks until after it commits or aborts.
  - Strict-2PL is that all exclusive (X) locks are held until the transaction reaches its commit or abort point. Shared (S) locks can still be released earlier, but this is less common as Strict-2PL is often seen as a lighter version of Rigorous 2PL, which holds all locks, shared or exclusive, until commit.

- Example Scenario

Let's say we have two transactions, T1 and T2, and a shared resource, Account A.

- **Transaction T1:** Deposit (Needs to write to A)
- Starts.
- Acquires an X-lock on A.
- Reads the value of A.
- Adds a deposit amount to A.
- Waits to commit: because it holds the X-lock, even though its read operation is complete.
- **Transaction T2**: Read Balance (Needs to read A)
- Starts.
- Tries to acquire an S-lock on A.
- T2 must wait: because T1 holds an X-lock on A.

Outcome

- When T1 successfully commits (or aborts), it releases the X-lock on A.
- Now, T2 can acquire the S-lock on A and read the updated balance.

Timestamp Ordering Protocol

- The Timestamp Ordering Protocol is used to order the transactions based on their Timestamps. The order of transaction is nothing but the ascending order of the transaction creation.
- The priority of the older transaction is higher that's why it executes first. To determine the timestamp of the transaction, this protocol uses system time or logical counter.
- The lock-based protocol is used to manage the order between conflicting pairs among transactions at the execution time. But Timestamp based protocols start working as soon as a transaction is created.
- Let's assume there are two transactions T1 and T2. Suppose the transaction T1 has entered the system at 007 times and transaction T2 has entered the system at 009 times. T1 has the higher priority, so it executes first as it is entered the system first.
- The timestamp ordering protocol also maintains the timestamp of last 'read' and 'write' operation on a data.

- TO protocol ensures serializability since the precedence graph is as follows:

Image: Precedence Graph for TS ordering

- TS protocol ensures freedom from deadlock that means no transaction ever waits.
- But the schedule may not be recoverable and may not even be cascade- free.

- In the Timestamp Ordering Protocol, transactions are assigned a unique timestamp upon arrival, and operations are allowed only if they follow the order of these timestamps to ensure serializability and data consistency. For example, with transactions T1 (timestamp 005) and T2 (timestamp 010), if T1 writes to data item A and T2 attempts to read A, the read is allowed because T1's write is older and T2's timestamp is greater than T1's read timestamp, according to the protocol's rules.

- How the Protocol Works

1. Timestamp Assignment: Each transaction (Ti) is assigned a unique timestamp (TS(Ti)) representing its arrival time or creation time.

2. Data Item Timestamps: Each data item (X) has a read timestamp (R-TS(X)) and a write timestamp (W-TS(X)), indicating the timestamp of the last transaction that read or wrote to it, respectively.

3. Operation Rules:

•Read(X) Operation:

- If TS(Ti) < W-TS(X), the operation is rejected because a newer transaction has already written to the data item.
- If TS(Ti) >= W-TS(X), the operation is allowed, and the R-TS(X) is updated to TS(Ti).

•Write(X) Operation:

- If TS(Ti) < R-TS(X), the operation is rejected because a newer transaction has already read the item.
- If TS(Ti) < W-TS(X), the operation is rejected, and the transaction Ti is rolled back because a newer transaction has already written to the item.
- If neither of the above conditions is met, the write operation is allowed, and W-TS(X) is updated to TS(Ti).

- Example
- Let's consider two transactions, T1 and T2, with TS(T1) = 5 and TS(T2) = 10, and a data item A with R-TS(A) = 0 and W-TS(A) = 0.

1. T1 Reads A:

- T1 issues Read(A).
- Since TS(T1) (5) is greater than W-TS(A) (0), the read is allowed.
- R-TS(A) is updated to 5.

2. T2 Writes A:

- T2 issues Write(A).
- Since TS(T2) (10) is greater than R-TS(A) (5), the operation is allowed.
- W-TS(A) is updated to 10.

3. T1 Writes A:

- T1 issues Write(A).
- Since TS(T1) (5) is less than W-TS(A) (10), the operation is rejected, and T1 is rolled back.

This example shows how the timestamp-based protocol uses timestamps to prevent conflicts and maintain the correct order of operations, ensuring serializability

Deadlocks

A deadlock is a condition where two or more transactions are waiting indefinitely for one another to give up locks. Deadlock is said to be one of the most feared complications in DBMS as no task ever gets finished and is in waiting state forever.

Deadlocks

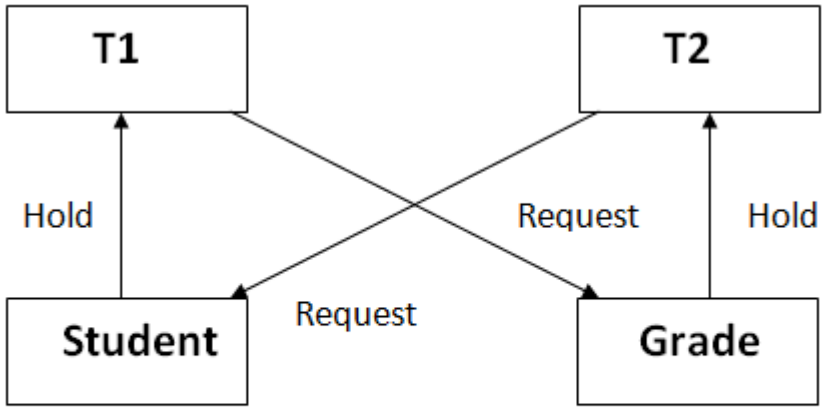


Figure: Deadlock in DBMS

In the student table, transaction T1 holds a lock on some rows and needs to update some rows in the grade table. Simultaneously, transaction T2 holds locks on some rows in the grade table and needs to update the rows in the Student table held by Transaction T1.

Now, the main problem arises. Now Transaction T1 is waiting for T2 to release its lock and similarly, transaction T2 is waiting for T1 to release its lock. All activities come to a halt state and remain at a standstill. It will remain in a standstill until the DBMS detects the deadlock and aborts one of the transactions.

Deadlocks

T1	T2
X(A) W(A)	
	X(B) W(B) X(A) W(A)
X(B) W(B)	

- Here, using strict 2PL, a lock is performed on A since there is a W(A).
- Now in T2, we again try to write on A, ie, W(A) but we have already locked A under T1.
- Hence there is a deadlock

- **Deadlock Avoidance**
- When a database is stuck in a deadlock state, then it is better to avoid the database rather than aborting or restating the database. This is a waste of time and resource.
- Deadlock avoidance mechanism is used to detect any deadlock situation in advance. A method like "wait for graph" is used for detecting the deadlock situation but this method is suitable only for the smaller database. For the larger database, deadlock prevention method can be used.

- **Deadlock Detection**
- In a database, when a transaction waits indefinitely to obtain a lock, then the DBMS should detect whether the transaction is involved in a deadlock or not. The lock manager maintains a Wait for the graph to detect the deadlock cycle in the database

Wait for Graph

- This is the suitable method for deadlock detection. In this method, a graph is created based on the transaction and their lock. If the created graph has a cycle or closed loop, then there is a deadlock.
- The wait for the graph is maintained by the system for every transaction which is waiting for some data held by the others. The system keeps checking the graph if there is any cycle in the graph.

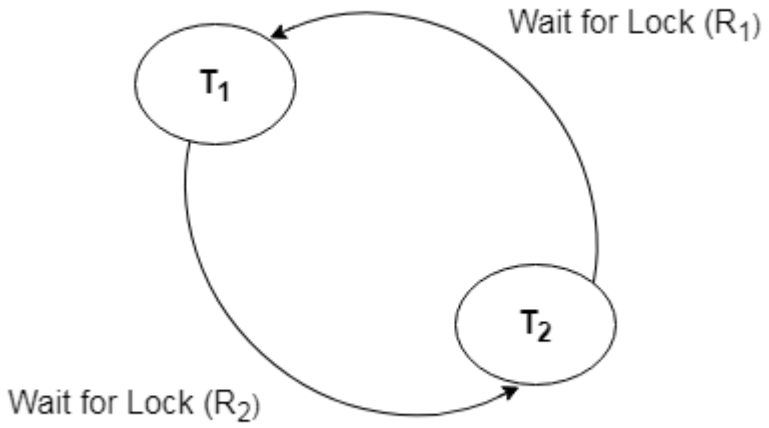
Deadlock Detection using wait for graph

Step 1: Draw one vertex for each transaction in the schedule.

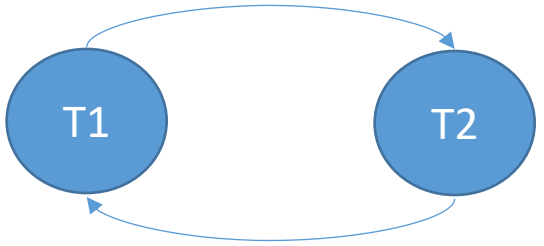
Step 2: An edge (Ti -> Tj) is added to the graph if and only if Ti is waiting for Tj.

Step 3: If the graph at any point of time has cycle, it implies deadlock.

Deadlock Detection using wait for graph

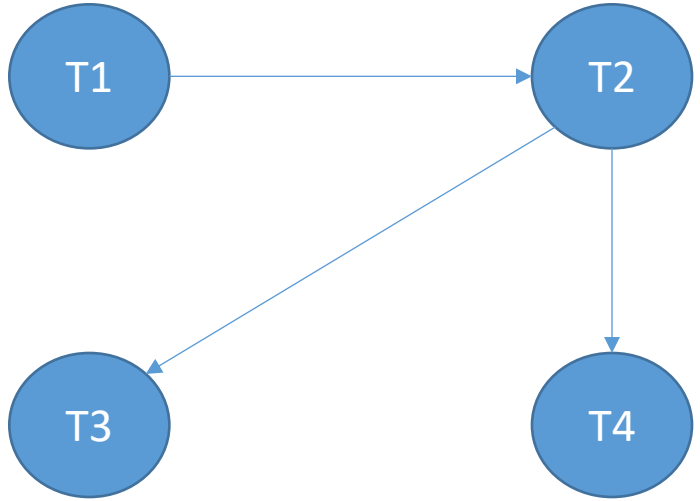


T1	T2
X(A)	
W(A)	
	X(B)
	W(B)
	X(A)
	W(A)
X(B)	
W(B)	



Deadlock Detection using wait for graph

T1	T2	T3	T4
X(A)	X(A) X(B)	X(A)	X(B)



Dealing with Deadlock and Starvation

Deadlock Prevention

Deadlock prevention method is suitable for a large database. If the resources are allocated in such a way that deadlock never occurs, then the deadlock can be prevented.

The Database management system analyzes the operations of the transaction whether they can create a deadlock situation or not. If they do, then the DBMS never allowed that transaction to be executed .

Protocols based on a timestamp:

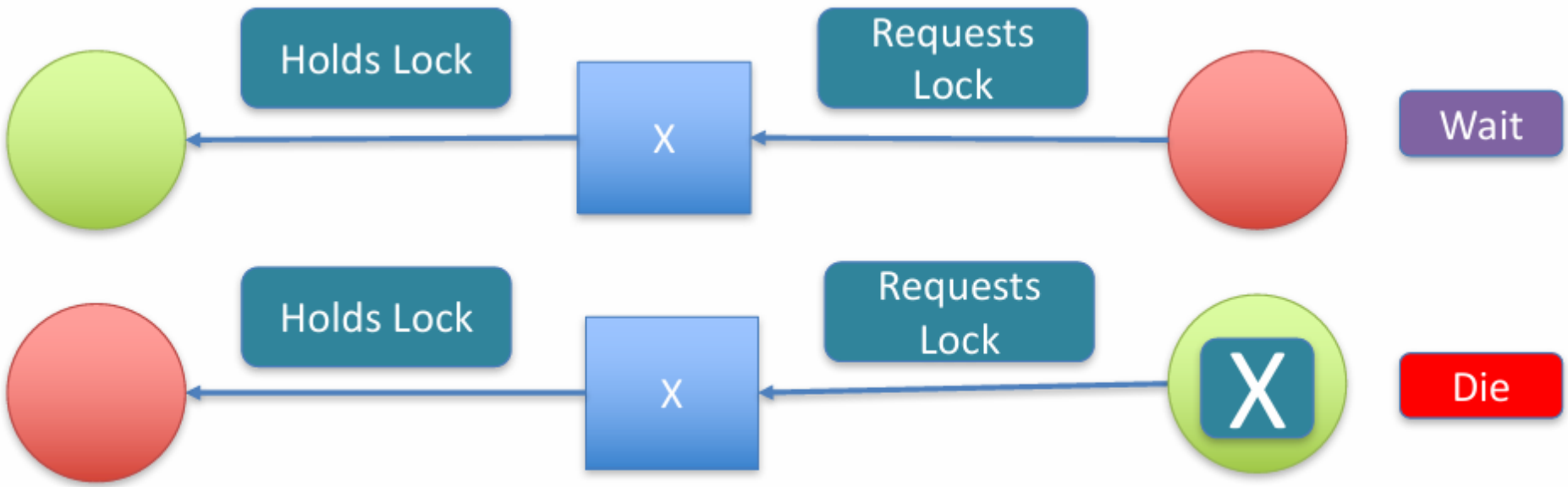
- Wait-die
- Wound-wait

- Wait-die :
- In this scheme, if a transaction requests for a resource which is already held with a conflicting lock by another transaction then the DBMS simply checks the timestamp of both transactions. It allows **the older transaction to wait until the resource is available for execution.**
- Let's assume there are two transactions Ti and Tj and let TS(T) is a timestamp of any transaction T. If T2 holds a lock by some other transaction and T1 is requesting for resources held by T2 then the following actions are performed by DBMS:
- Check if TS(Ti) < TS (Tj) - If Ti is the older transaction and Tj has held some resource, then Ti is allowed to wait until the data-item is available for execution. That means if the older transaction is waiting for a resource which is locked by the younger transaction, then the older transaction is allowed to wait for resource until it is available.
- Check if TS(T_i) < TS(Tj) - If Ti is older transaction and has held some resource and if Tj is waiting for it, then Tj is killed and restarted later with the random delay but with the same timestamp.

Wait-Die

T_{old} is allowed to ***wait*** for T_{new}

T_{new} will ***die*** when it waits for T_{old}

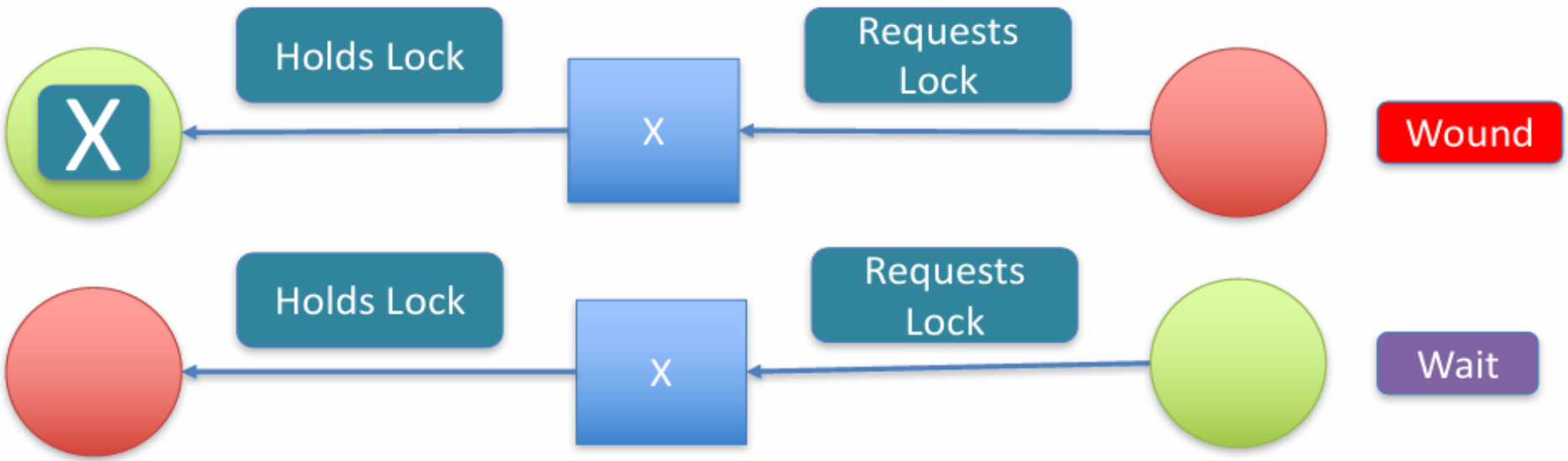


- Wound wait scheme
- In wound wait scheme, if the older transaction requests for a resource which is held by the younger transaction, then older transaction forces younger one to kill the transaction and release the resource. After the minute delay, the younger transaction is restarted but with the same timestamp.
- If the older transaction has held a resource which is requested by the Younger transaction, then the younger transaction is asked to wait until older releases it.

Wound Wait

T_{old} will wound T_{new}

T_{new} waits for T_{old}



- wait-die :
- When transaction Ti requests a data item currently held by Tj Ti is allowed to wait only if it has a timestamp smaller than that of Tj (that is, Ti is older than Tj). Otherwise, Ti is rolled back (dies). This is wait-die.
- wound-wait:
- When transaction Ti requests a data item currently held by Tj , Ti is allowed to wait only if it has a timestamp larger than that of Tj (that is, Ti is younger than Tj). Otherwise, Tj is rolled back (Tj is wounded by Ti). This is wound-wait.

- **Which concurrency control protocols ensure conflict serializability and freedom from the deadlock?**

  A) 2-phase locking
  B) Time-stamp Ordering
  C) Both 2-phase locking and Time-stamp ordering
  D) None of the above

  ANSWER:- B
  Explanation :2PL, i.e., 2 Phase Locking, is a concurrency control method that guarantees serializability. This protocol uses locks applied to data by a transaction, which blocks other transactions from accessing the same data during the transaction's life. Mutual blocking of 2 or more transactions may lead to deadlock in 2PL.

- **Which among the following statements is/are incorrect?**
  1) A schedule following a strict two-phase locking protocol is conflict serializable and recoverable.
  2) Checkpoint in schedules inserted to ensure recoverability.

  A) Only 1
  B) Only 2
  C) Both 1 and 2
  D) None of the above

  ANSWER:- B
  Explanation :A basic two-phase locking protocol ensures only conflict serializability. In contrast, a strict two-phase locking protocol ensures conflict serializability and recoverability. So statement (1) is correct. i.e., checkpoints are inserted to minimize the task of undo-redo in recoverability. And hence statement (2) is not correct. So Option (B) is correct.